

⊕ Async JS

→ by default in JS → Synchronous
→ Single threaded
 ↳ one call stack()
 ↳ one main thread

exact order
↑
one statement at a time

→ Execution Context

env. containing everything to run that code
↳ execute one line of code at a time

→ console.log → 1
→ console.log → 2

Call stack Memory heap

→ Executional context has 2 phases: { every fn. call() has its own EC }

→ Memory creation phase → mem is allocated for var, ...

→ execution phase → executing one line at a time.

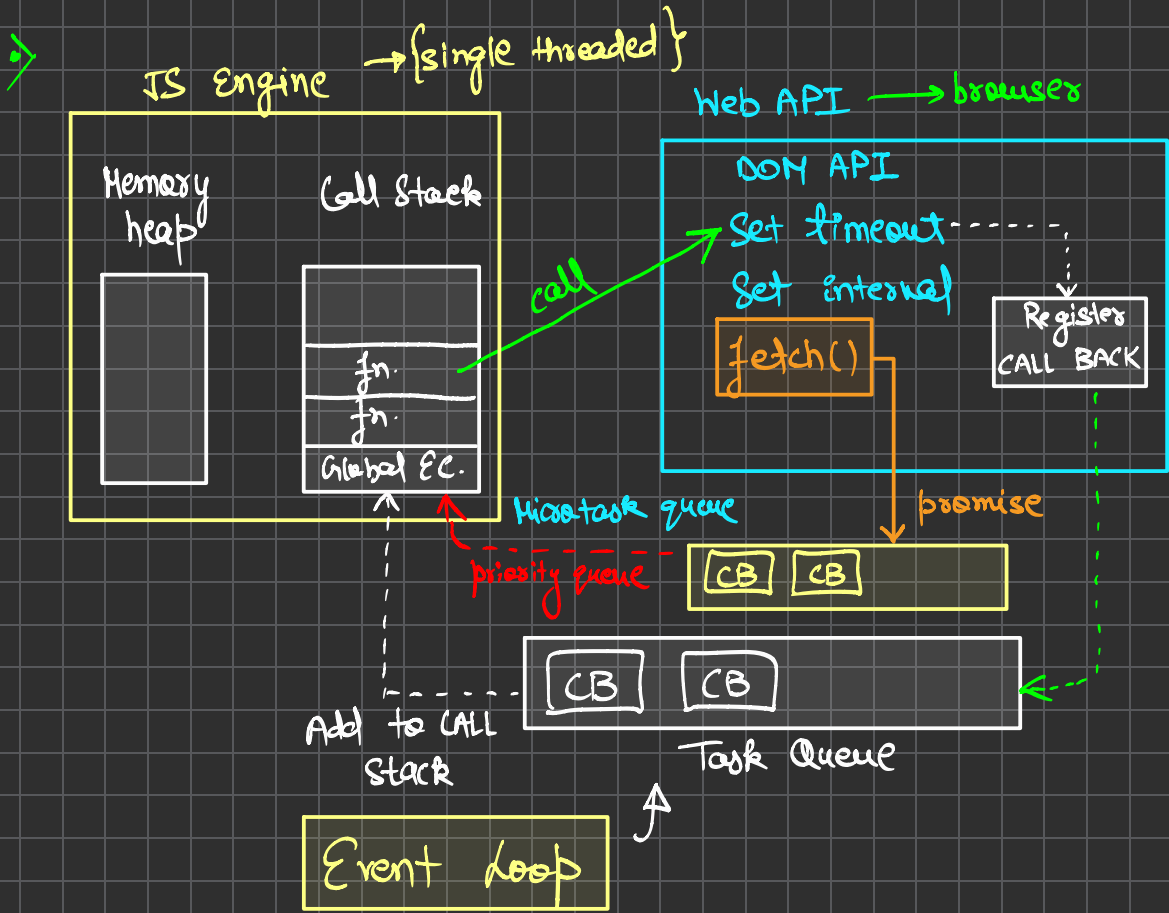
Blocking code & Non blocking code

↓
block flow of program

↓
Read file sync

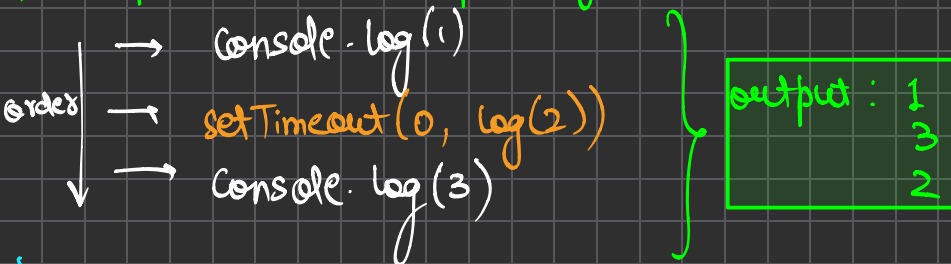
↓
doesn't block of execution

↓
Read file async



#Example:

➤ Suppose we are printing 3 no.s → 1, 2 & 3



➤ 2 is printed after 3, becoz 2 is registered and then goes to callback task queue, meanwhile 3 is printed and at last 2 is printed.

response = fetch('something')

also private

Data:

{
 onfulfilled[]
 onRejection[]
}

both are private fields

not directly accessible

Web Browser/node

Network request

ex: 404

Global

response